ENGINEERING LECTURE · PYTHON AND JAVA

Designing for Change

Inheritance vs Composition · SOLID · Design Patterns

[Presenter name] · [Team, date]

Every example in Python 3.10+ and Java 21, ported from github.com/himasz/design-patterns-c-sharp

Agenda

ABOUT 90 MINUTES

01

Inheritance vs Composition

≈ 15 min

Two ways to reuse behavior, and why one usually wins.

02

SOLID

≈ 25 min

Five principles, each with a before and after.

03

Design Patterns

≈ 40 min

Eleven patterns from the C# repo, in Python and Java.

04

Wrap-up

≈ 10 min + lab

Language shortcuts, pitfalls, takeaways and exercises.

GOAL Leave with a shared vocabulary, and the habit of making the next change cheap.

Why Python, with Java alongside

THE LANGUAGE CHOICE

Everyone reads it

The team knows Python, so we spend the hour on design, not syntax.

Java shows the shape

The same example in Java proves the design isn’t a Python trick.

Less ceremony

Python hides less behind boilerplate; Java shows what types make explicit.

WATCH OUT Python doesn’t enforce contracts at runtime: run mypy or pyright in CI. Java’s compiler does it for you.

Reading the C# repo

CHEAT SHEET

In the C# repo	In Java	In Python
interface ICar { ... }	interface Car { ... }	typing.Protocol, or abc.ABC
abstract class + method	the same	abc.ABC + @abstractmethod
switch expression on an enum	switch, or a Map	a dict, or match (3.10+)
method overloads	the same	defaults, keyword args, *args
private readonly field	private final	_leading_underscore
static Instance property	holder class, or enum	@functools.cache, or a module
event, INotifyPropertyChanged	listener interface	a list of callbacks
WeakReference<T>	WeakReference<T>	weakref.ref, weakref.WeakSet

Interfaces, part 1: ABC

PYTHON PRIMER

PYTHON

```python
from abc import ABC, abstractmethod

class SuperPower(ABC):
    @abstractmethod
    def exercise_power(self) -> str: ...

class Fly(SuperPower):          # must inherit
    def exercise_power(self) -> str:
        return "Up, Up, and Away!"

power: SuperPower = Fly()
# SuperPower()  -> TypeError: can't instantiate
# A subclass that forgets exercise_power()
#   -> TypeError the moment you create one
```

JAVA

```java
abstract class SuperPower {
    abstract String exercisePower();
}

class Fly extends SuperPower {       // must extend
    String exercisePower() {
        return "Up, Up, and Away!";
    }
}

SuperPower power = new Fly();
// new SuperPower()  -> compile error
// A subclass that forgets exercisePower()
//   -> compile error too
```

NOMINAL

Classes must inherit it, as Java classes must extend an abstract class.

CHECKED

Python refuses an incomplete subclass when you create one; javac won't compile.

USE IT FOR

A base class that also carries shared code, like Template Method.

Interfaces, part 2: Protocol

PYTHON PRIMER

PYTHON

```python
from typing import Protocol

class SuperPower(Protocol):
    def exercise_power(self) -> str: ...

class Fly:                      # no base class at all
    def exercise_power(self) -> str:
        return "Up, Up, and Away!"

power: SuperPower = Fly()
# Fits by shape ("duck typing"), and
# mypy / pyright verify the shape for you.
# Great for code you don't own.
```

JAVA

```java
interface SuperPower {
    String exercisePower();
}

class Fly implements SuperPower {  // must say so
    public String exercisePower() {
        return "Up, Up, and Away!";
    }
}

SuperPower power = new Fly();
// Java interfaces are nominal: a class that has
// the method but no `implements` does NOT fit.
// Nothing in Java matches Protocol's duck typing.
```

STRUCTURAL

Any class with the right methods fits; it never names the Protocol.

JAVA HAS NONE

Java's interfaces are nominal: no implements, no fit. TypeScript's match Python.

CHECKED BY

mypy or pyright, before the code runs. Nothing is checked at runtime.

PART 01

Inheritance vs Composition

Two ways to reuse behavior: be something, or have something.

01

Two ways to reuse behavior

PART 1

INHERITANCE · IS-A

Be a specialised parent

```
class FlyingHero(Hero): ...
```

```
class FlyingHero extends Hero {…}
```

- Reuse by extending a parent
- The child sees the parent’s internals
- Fixed when the class is written
- One family tree to fit into

COMPOSITION · HAS-A

Hold a part that does the job

```
self.power = Fly()
```

```
this.power = new Fly();
```

- Reuse by delegating to a part
- Depends only on the part’s interface
- Parts can be swapped at runtime
- Mix as many parts as you need

The inheritance trap

PART 1 · PROBLEM

PYTHON

```python
class Hero:
    def act(self) -> str: return "Run away"

class FlyingHero(Hero):
    def act(self) -> str: return "Up, Up, and Away!"

class FightingHero(Hero):
    def act(self) -> str: return "KaBaam!"

# Needs both? Yet another subclass...
class FlyingFightingHero(FlyingHero, FightingHero):
    pass

print(FlyingFightingHero().act())
# → 'Up, Up, and Away!'
#    the MRO picked one; KaBaam! is lost
```

JAVA

```java
class Hero {
    String act() { return "Run away"; }
}
class FlyingHero extends Hero {
    String act() { return "Up, Up, and Away!"; }
}
class FightingHero extends Hero {
    String act() { return "KaBaam!"; }
}
// Needs both? Java won't even compile this:
// class FlyingFightingHero
//     extends FlyingHero, FightingHero {}
// So you copy one behavior in by hand:
class FlyingFightingHero extends FlyingHero {
    // ...paste FightingHero's act() here?
}
```

WHAT HAPPENS

Each mix of behaviors needs its own subclass: 3 powers, 7 hero classes.

PYTHON

Multiple inheritance runs, but the MRO quietly picks one act().

JAVA

One superclass only: the combined hero can’t even be declared.

The same heroes, composed

PYTHON

```python
class SuperPower(Protocol):
    def exercise(self) -> str: ...

class Fly:
    def exercise(self): return "Up, Up, and Away!"
class Fight:
    def exercise(self): return "KaBaam!"

class Hero:
    def __init__(self, *powers: SuperPower) -> None:
        self.powers = list(powers)     # has-a
    def act(self) -> str:
        return " / ".join(
            p.exercise() for p in self.powers)

print(Hero(Fly(), Fight()).act())
# → 'Up, Up, and Away! / KaBaam!'
```

JAVA

```java
interface SuperPower { String exercise(); }

class Hero {
    private final List<SuperPower> powers;
    Hero(SuperPower... powers) {        // has-a
        this.powers = List.of(powers);
    }
    String act() {
        return powers.stream()
            .map(SuperPower::exercise)
            .collect(Collectors.joining(" / "));
    }
}
// in main(): a one-method interface takes a lambda
SuperPower fly = () -> "Up, Up, and Away!";
new Hero(fly, () -> "KaBaam!").act();
// → "Up, Up, and Away! / KaBaam!"
```

HAS-A

A Hero holds powers instead of being one. Any mix, no new classes.

JAVA BONUS

SuperPower has one method, so a lambda is a complete implementation.

PREVIEW

This is the Strategy pattern, from the repo's Heroes example.

Inheritance exposes everything

PART 1

PYTHON

```python
class Stack(list):          # Stack IS-A list
    def push(self, item) -> None:
        self.append(item)

s = Stack()
s.push(1)
s.push(2)
s.insert(0, 99)   # allowed: LIFO rule broken
s.sort()          # also allowed
s.clear()         # ...every list method works
```

JAVA

```java
class Stack<T> extends ArrayList<T> {   // IS-A list
    void push(T item) { add(item); }
}
// in main():
var s = new Stack<Integer>();
s.push(1);
s.push(2);
s.add(0, 99);     // allowed: LIFO rule broken
s.sort(null);     // also allowed
s.clear();        // ...every List method works

// The JDK made this exact mistake:
// java.util.Stack extends Vector.
```

IS-A

Subclass list and Stack inherits insert, sort, clear and the rest.

THE JDK DID IT

java.util.Stack extends Vector, so you can insert into its middle.

THE RESULT

The class can’t protect its own rule: last in, first out.

Composition shows only what it means

PART 1

PYTHON

```python
class Stack:                    # Stack HAS-A list
    def __init__(self) -> None:
        self._items: list = []
    def push(self, item) -> None:
        self._items.append(item)
    def pop(self):
        return self._items.pop()
    def __len__(self) -> int:
        return len(self._items)

s = Stack()
s.push(1)
# s.insert(0, 99)  -> AttributeError. Good!
```

JAVA

```java
class Stack<T> {                        // HAS-A a deque
    private final Deque<T> items =
        new ArrayDeque<>();
    void push(T item) { items.push(item); }
    T pop() { return items.pop(); }
    int size() { return items.size(); }
}
// in main():
var s = new Stack<Integer>();
s.push(1);
// s.add(0, 99);  -> compile error. Good!
```

HAS-A

The list or deque is a private part; only push, pop and size exist.

SWAPPABLE

Change the list for a deque inside; no caller can tell.

CAUGHT EARLY

Python raises AttributeError; Java won't compile the call.

The trade-offs side by side

PART 1

	Inheritance	Composition
Relationship	is-a	has-a, uses-a
Coupling	high: child knows parent internals	low: only the part’s interface
Change behavior	when the class is written	at runtime: swap the part
Combine behaviors	a subclass per combination	mix parts freely
Testing	through the whole hierarchy	parts alone; inject fakes
Up-front cost	less code	a little more wiring

THE RULE OF THUMB

“Favor object composition over class inheritance.”

Gamma, Helm, Johnson & Vlissides, *Design Patterns* (1994)

When inheritance is the right tool

PART 1

It really is-a

A subclass can stand in for its parent everywhere (Liskov holds).

The tree stays shallow

One or two levels. Deep trees grow fragile base classes.

A framework asks for it

Exceptions, test base classes, framework hooks: the base is the hook.

You’re filling in a template

The parent owns the algorithm; the child fills in the steps.

Otherwise: compose, and share code through small helper objects or plain functions.

PART 02

SOLID

Five principles for code that’s easy to change, each with a before and after.

02

SOLID at a glance

PART 2

S — **Single Responsibility**
One reason to change: a class answers to one actor.

O — **Open/Closed**
Add behavior by adding code, not by editing working code.

L — **Liskov Substitution**
A subtype must keep every promise its base type makes.

I — **Interface Segregation**
Many small, role-sized interfaces beat one fat one.

D — **Dependency Inversion**
Depend on abstractions; plug the details in from outside.

Collected by Robert C. Martin around 2000; the acronym was coined by Michael Feathers.

S — Single Responsibility: before

PYTHON

```python
class SalesReport:
    def __init__(self, orders):
        self.orders = orders

    def total(self) -> float:        # finance
        return sum(o.price for o in self.orders)

    def to_html(self) -> str:        # design team
        return f"<h1>{self.total():.2f}</h1>"

    def save(self, path: str):       # ops / storage
        with open(path, "w") as f:
            f.write(self.to_html())
```

JAVA

```java
record SalesReport(List<Order> orders) {
    double total() {                    // finance
        return orders.stream()
            .mapToDouble(Order::price).sum();
    }
    String toHtml() {                   // design team
        return "<h1>%.2f</h1>".formatted(total());
    }
    void save(Path p) throws IOException { // ops
        Files.writeString(p, toHtml());
    }
}
```

THE PRINCIPLE

A class should have one reason to change: it answers to one group of people.

THE SMELL

Finance, the design team and ops all edit this one class.

JAVA

The same shape: one record carrying three unrelated jobs.

S — Single Responsibility: after

PYTHON

```python
class SalesReport:                    # finance
    def __init__(self, orders):
        self.orders = orders
    def total(self) -> float:
        return sum(o.price for o in self.orders)

class HtmlFormatter:                  # design team
    def format(self, r: SalesReport) -> str:
        return f"<h1>{r.total():.2f}</h1>"

class FileStore:                      # ops / storage
    def save(self, path: str, text: str):
        with open(path, "w") as f:
            f.write(text)
```

JAVA

```java
record SalesReport(List<Order> orders) { // finance
    double total() {
        return orders.stream()
            .mapToDouble(Order::price).sum();
    }
}
class HtmlFormatter {                    // design team
    String format(SalesReport r) {
        return "<h1>%.2f</h1>".formatted(r.total());
    }
}

class FileStore {                      // ops / storage
    void save(Path p, String text)
            throws IOException {
        Files.writeString(p, text);
    }
}
```

THE FIX

One class per actor, so each has exactly one reason to change.

A QUICK TEST

Describe the class without saying “and”.

JAVA

A record holds the data; small classes do each other job.

O — Open/Closed: before

PYTHON

```python
def checkout(total: float, kind: str) -> float:
    if kind == "regular":
        return total
    elif kind == "student":
        return total * 0.9
    elif kind == "vip":
        return total * 0.8 - 5
    raise ValueError(kind)

# New customer type?
#   → edit, re-test, re-deploy checkout()
```

JAVA

```java
static double checkout(double total, String kind) {
    if (kind.equals("regular")) {
        return total;
    } else if (kind.equals("student")) {
        return total * 0.9;
    } else if (kind.equals("vip")) {
        return total * 0.8 - 5;
    }
    throw new IllegalArgumentException(kind);
}

// New customer type?
//   → edit, re-test, re-deploy checkout()
```

THE PRINCIPLE

Open for extension, closed for modification: add code, don’t edit it.

THE SMELL

Every new customer type edits, re-tests and redeploys checkout().

IN THE REPO

SimplePizzaFactory’s switch has the same problem.

O — Open/Closed: after

PYTHON

```python
class Discount(Protocol):
    def apply(self, total: float) -> float: ...

class Regular:
    def apply(self, total): return total
class Student:
    def apply(self, total): return total * 0.9
class Vip:
    def apply(self, total): return total * 0.8 - 5

def checkout(total: float, d: Discount) -> float:
    return d.apply(total)       # never edited again

# New customer type?  → add a class
checkout(100, Student())        # 90.0
```

JAVA

```java
interface Discount {
    double apply(double total);

    // one implementation per rule (lambdas here)
    Discount REGULAR = t -> t;
    Discount STUDENT = t -> t * 0.9;
    Discount VIP     = t -> t * 0.8 - 5;
}

static double checkout(double total, Discount d) {
    return d.apply(total);      // never edited again
}

// New customer type?  → add a Discount
checkout(100, Discount.STUDENT);   // 90.0
```

THE FIX

checkout() depends on a Discount interface and never changes again.

PYTHON

One small class per rule, matched by a Protocol.

JAVA

Discount has one method, so each rule can be a lambda.

L — Liskov Substitution: before

PYTHON

```python
class Character(Protocol):
    def chase(self) -> int: ...
    def flee(self) -> int: ...
    def attack(self) -> int: ...

class BadGuyAdapter:                    # from the repo
    def __init__(self, bad_guy: BadGuy):
        self._g = bad_guy
    def chase(self) -> int: return self._g.drive()
    def attack(self) -> int: return self._g.shoot()
    def flee(self) -> int:
        raise Exception("Bad guys don't flee!")

def retreat(party: list[Character]) -> int:
    return sum(c.flee() for c in party)

# retreat([BadGuyAdapter(BadGuy())])  -> crash
```

JAVA

```java
interface GameCharacter {
    int chase(); int flee(); int attack();
}
class BadGuyAdapter implements GameCharacter {
    private final BadGuy g;
    BadGuyAdapter(BadGuy g) { this.g = g; }
    public int chase()  { return g.drive(); }
    public int attack() { return g.shoot(); }
    public int flee() {
        throw new UnsupportedOperationException(
            "Bad guys don't flee!");
    }
}
static int retreat(List<GameCharacter> party) {
    return party.stream()
        .mapToInt(GameCharacter::flee).sum(); // boom
}
```

THE PRINCIPLE

A subtype must work wherever its base type is used. No surprise errors.

FROM THE REPO

BadGuyAdapter claims to be a character, then refuses to flee.

THE RESULT

Any loop over characters crashes as soon as a bad guy joins.

L — Liskov Substitution: after

PYTHON

```python
class Combatant(Protocol):
    def chase(self) -> int: ...
    def attack(self) -> int: ...

class Escaper(Combatant, Protocol):
    def flee(self) -> int: ...

class BadGuyAdapter:          # no flee() promised
    def __init__(self, bad_guy: BadGuy):
        self._g = bad_guy
    def chase(self) -> int: return self._g.drive()
    def attack(self) -> int: return self._g.shoot()

def retreat(party: list[Escaper]) -> int:
    return sum(c.flee() for c in party)

# retreat([BadGuyAdapter(BadGuy())])
#   -> mypy error: BadGuyAdapter is not an Escaper
```

JAVA

```java
interface Combatant { int chase(); int attack(); }
interface Escaper extends Combatant { int flee(); }

class BadGuyAdapter implements Combatant { // no flee
    private final BadGuy g;
    BadGuyAdapter(BadGuy g) { this.g = g; }
    public int chase()  { return g.drive(); }
    public int attack() { return g.shoot(); }
}

static int retreat(List<Escaper> party) {
    return party.stream()
        .mapToInt(Escaper::flee).sum();
}

// retreat(List.of(new BadGuyAdapter(new BadGuy())));
//   -> javac error: incompatible types
```

THE FIX

Split the contract, and promise only what you can keep.

PYTHON

mypy rejects a BadGuyAdapter where an Escaper is needed.

JAVA

javac rejects it too: the bug never reaches runtime.

I — Interface Segregation: before

SOLID 4/5

PYTHON

```python
class IngredientFactory(ABC):     # the repo's shape
    @abstractmethod
    def make_dough(self) -> str: ...
    @abstractmethod
    def make_sauce(self) -> str: ...
    @abstractmethod
    def make_cheese(self) -> str: ...
    @abstractmethod
    def make_sausage(self) -> str: ...
    @abstractmethod
    def make_pepperoni(self) -> str: ...

class VeggieKitchen(IngredientFactory):
    # make_dough / make_sauce / make_cheese ...
    def make_sausage(self) -> str:
        raise NotImplementedError   # forced to
    def make_pepperoni(self) -> str:
        raise NotImplementedError   # forced to
```

JAVA

```java
interface IngredientFactory {   // the repo's shape
    String makeDough();
    String makeSauce();
    String makeCheese();
    String makeSausage();
    String makePepperoni();
}

class VeggieKitchen implements IngredientFactory {
    // makeDough / makeSauce / makeCheese ...
    public String makeSausage() {      // forced to
        throw new UnsupportedOperationException();
    }
    public String makePepperoni() {    // forced to
        throw new UnsupportedOperationException();
    }
}
```

THE PRINCIPLE

Clients shouldn’t depend on methods they don’t use.

THE REPO’S SHAPE

IIngredientFactory makes every kitchen implement sausage and pepperoni.

THE SMELL

Methods whose only honest body is “not implemented”.

I — Interface Segregation: after

SOLID 4/5

PYTHON

```python
class BaseIngredients(Protocol):
    def make_dough(self) → str: ...
    def make_sauce(self) → str: ...
    def make_cheese(self) → str: ...

class MeatIngredients(Protocol):
    def make_sausage(self) → str: ...
    def make_pepperoni(self) → str: ...

class VeggieKitchen:          # only what it does
    def make_dough(self): return "wheat"
    def make_sauce(self): return "tomato"
    def make_cheese(self): return "vegan"

def cheese_pizza(k: BaseIngredients) → list[str]:
    return [k.make_dough(), k.make_sauce(),
            k.make_cheese()]
```

JAVA

```java
interface BaseIngredients {
    String makeDough();
    String makeSauce();
    String makeCheese();
}
interface MeatIngredients {
    String makeSausage(); String makePepperoni();
}

class VeggieKitchen implements BaseIngredients {
    public String makeDough()  { return "wheat"; }
    public String makeSauce()  { return "tomato"; }
    public String makeCheese() { return "vegan"; }
}
static List<String> cheesePizza(BaseIngredients k) {
    return List.of(k.makeDough(), k.makeSauce(),
                   k.makeCheese());
}
```

THE FIX

Small, role-sized interfaces. A kitchen implements only what it does.

PYTHON

Protocols cost nothing to add: classes don’t declare them.

JAVA

A meat kitchen would implement both: implements A, B.

D — Dependency Inversion: before

SOLID 5/5

PYTHON

```python
class PostgresRepo:
    def save(self, order: dict) -> None:
        ...                     # INSERT INTO orders

class OrderService:             # high-level policy
    def __init__(self) -> None:
        self._repo = PostgresRepo()  # hard-wired
    def place(self, order: dict) -> None:
        self._repo.save(order)

# Testing place() now needs a real database,
# and switching storage means editing the
# business logic.
```

JAVA

```java
class PostgresRepo {
    void save(Order order) {
        // INSERT INTO orders ...
    }
}

class OrderService {                // high-level policy
    private final PostgresRepo repo =
        new PostgresRepo();         // hard-wired
    void place(Order order) { repo.save(order); }
}

// Testing place() now needs a real database,
// and switching storage means editing the
// business logic.
```

THE PRINCIPLE

High-level code depends on an abstraction; details plug in from outside.

THE SMELL

Business logic constructs its own database class.

THE COST

No test without a database, and storage changes edit business code.

D — Dependency Inversion: after

SOLID 5/5

PYTHON

```python
class OrderRepository(Protocol):    # abstraction
    def save(self, order: dict) -> None: ...

class OrderService:
    def __init__(self, repo: OrderRepository):
        self._repo = repo           # injected
    def place(self, order: dict) -> None:
        self._repo.save(order)

class InMemoryRepo:                 # test double
    def __init__(self) -> None:
        self.rows: list[dict] = []
    def save(self, order: dict) -> None:
        self.rows.append(order)

OrderService(InMemoryRepo())        # in tests
OrderService(PostgresRepo())        # in production
```

JAVA

```java
interface OrderRepository {          // abstraction
    void save(Order order);
}
class OrderService {
    private final OrderRepository repo;
    OrderService(OrderRepository repo) {
        this.repo = repo;            // injected
    }
    void place(Order order) { repo.save(order); }
}
class InMemoryRepo implements OrderRepository {
    final List<Order> rows = new ArrayList<>();
    public void save(Order o) { rows.add(o); }
}

// new OrderService(new InMemoryRepo())  in tests
// new OrderService(new PostgresRepo())  in prod
```

THE FIX

OrderService asks for any OrderRepository; the caller chooses which.

IN THE REPO

Every pizza store takes an IPizzaFactory in its constructor.

JAVA

This is Spring’s constructor injection, without the framework.

Spotting SOLID problems in a code review

PART 2

	The smell you'll notice	The usual fix
S	One change touches unrelated features; names like Manager	Split by who asks for changes
O	A growing if/elif or switch on a type code	Polymorphism: Strategy, Decorator
L	Overrides that throw, do nothing or need type checks	A narrower contract, or compose
I	Methods that only throw "not implemented"	Split into role-sized interfaces
D	Business code constructs infrastructure	Inject an interface; build at the edge

PART 03

Design Patterns

Named, reusable solutions to design problems that keep coming back.

03

What a design pattern is

PART 3

Not a library to install: a shared vocabulary for design. Each one has four parts.

Name

A word the team shares: “let’s use a Strategy here.”

Problem

The situation that keeps recurring, and when to apply it.

Solution

The roles and how they collaborate, not concrete code.

Consequences

The trade-offs you buy: flexibility for indirection.

23 patterns in the 1994 Gang of Four catalogue, in three families: creational, structural and behavioral.

The eleven patterns in the repo

PART 3

CREATIONAL

How objects get made

Singleton
One instance, one access point

Simple Factory
One place picks the class

Factory Method
Subclasses pick the class

Abstract Factory
Families of matching objects

STRUCTURAL

How objects fit together

Adapter
Make an interface fit

Decorator
Add behavior by wrapping

Facade
One simple front door

BEHAVIORAL

How objects share work

Strategy
Swap an algorithm

Observer
Notify every subscriber

Command
A request as an object

Template Method
Fixed steps, varying details

Simple Factory isn’t in the GoF catalogue; it’s the stepping stone to the two factory patterns.

Singleton: the classic version

CREATIONAL

PYTHON

```python
import threading

class Settings:
    _instance: "Settings | None" = None
    _lock = threading.Lock()
    some_value: int

    def __new__(cls) -> "Settings":
        if cls._instance is None:      # fast path
            with cls._lock:            # serialize
                if cls._instance is None:  # again
                    obj = super().__new__(cls)
                    obj.some_value = 0
                    cls._instance = obj
        return cls._instance

assert Settings() is Settings()
```

JAVA

```java
final class Settings {
    private static volatile Settings instance;
    int someValue = 0;

    private Settings() {}

    static Settings getInstance() {
        if (instance == null) {          // fast path
            synchronized (Settings.class) {
                if (instance == null) {  // again
                    instance = new Settings();
                }
            }
        }
        return instance;
    }
}
// Settings.getInstance() == Settings.getInstance()
```

INTENT

One instance and one global access point.

THREAD SAFETY

Check, lock, check again. Java also needs volatile on the field.

THE CATCH

It’s global state: hidden dependencies and tests that share it.

Singleton: let the language do it

CREATIONAL

PYTHON

```python
from functools import cache

class Settings:
    def __init__(self) -> None:
        self.some_value = 0

@cache                          # built on first call
def get_settings() -> Settings:
    return Settings()

assert get_settings() is get_settings()

# Simplest of all: a module is imported once.
#   settings.py:   some_value = 0
#   elsewhere:     import settings
```

JAVA

```java
// Lazy holder: the JVM loads Holder on first use,
// and class loading is thread-safe.
final class Settings {
    int someValue = 0;
    private Settings() {}
    private static class Holder {
        static final Settings INSTANCE =
            new Settings();
    }
    static Settings get() { return Holder.INSTANCE; }
}

// Simplest of all (Effective Java, item 3):
enum Config { INSTANCE; int someValue = 0; }

// Settings.get() == Settings.get()
// Config.INSTANCE.someValue = 42;
```

PYTHON

@cache builds it on first call; a module is already a singleton.

JAVA

The holder class is lazy and thread-safe; an enum is simplest of all.

BETTER STILL

Create one at startup and pass it in: Dependency Inversion again.

Simple Factory

CREATIONAL

PYTHON

```python
from typing import Callable
from pizza import (Pizza, PizzaType, CheesePizza,
                   PepperoniPizza, SausagePizza)

Maker = Callable[[], Pizza]          # a type alias

class SimplePizzaFactory:
    _registry: dict[PizzaType, Maker] = {
        PizzaType.CHEESE: CheesePizza,
        PizzaType.PEPPERONI: PepperoniPizza,
        PizzaType.SAUSAGE: SausagePizza,
    }
    def create_pizza(self, kind: PizzaType) -> Pizza:
        return self._registry[kind]()   # KeyError

factory = SimplePizzaFactory()
pizza = factory.create_pizza(PizzaType.CHEESE)
```

JAVA

```java
class SimplePizzaFactory {
    static final Map<PizzaType, Supplier<Pizza>>
        REGISTRY = Map.of(
            PizzaType.CHEESE, CheesePizza::new,
            PizzaType.PEPPERONI, PepperoniPizza::new,
            PizzaType.SAUSAGE, SausagePizza::new);

    Pizza createPizza(PizzaType kind) {
        return REGISTRY.get(kind).get();
    }   // a missing key → NullPointerException
}

// in main(): CheesePizza::new is a constructor
// reference, Java's "class as a callable"
var factory = new SimplePizzaFactory();
var pizza = factory.createPizza(PizzaType.CHEESE);
```

INTENT

One place decides which class to build; callers never name constructors.

CLASS AS A FUNCTION

Python stores the class itself; Java stores CheesePizza::new.

MISSING KEY

Python raises KeyError; in Java, Map.get gives null and .get() throws.

Factory Method

CREATIONAL

PYTHON

```python
class PizzaStore(ABC):
    def order_pizza(self, kind: PizzaType) -> Pizza:
        p = self.create_pizza(kind)  # factory method
        p.prepare(); p.bake(); p.cut(); p.box()
        return p

    @abstractmethod
    def create_pizza(self, kind) -> Pizza: ...

class NewYorkStore(PizzaStore):
    def create_pizza(self, kind: PizzaType) -> Pizza:
        return Pizza(kind, dough="thin")

class ChicagoStore(PizzaStore):
    def create_pizza(self, kind: PizzaType) -> Pizza:
        return Pizza(kind, dough="deep dish")

pizza = ChicagoStore().order_pizza(PizzaType.SAUSAGE)
```

JAVA

```java
abstract class PizzaStore {
    Pizza orderPizza(PizzaType kind) {
        var p = createPizza(kind); // factory method
        p.prepare(); p.bake(); p.cut(); p.box();
        return p;
    }
    abstract Pizza createPizza(PizzaType kind);
}
class NewYorkStore extends PizzaStore {
    Pizza createPizza(PizzaType kind) {
        return new Pizza(kind, "thin");
    }
}
class ChicagoStore extends PizzaStore {
    Pizza createPizza(PizzaType kind) {
        return new Pizza(kind, "deep dish");
    }
}
```

INTENT

The base class runs the process; subclasses decide what gets created.

IN THE REPO

SimplePizzaStoreBase.OrderPizza() calls the abstract CreatePizza().

USAGE

ChicagoStore().order_pizza(SAUSAGE) gives a deep dish, baked and boxed.

Abstract Factory: the families

CREATIONAL

PYTHON

```python
class IngredientFactory(Protocol):     # one family
    def dough(self) → str: ...
    def sauce(self) → str: ...
    def cheese(self) → str: ...

class NewYorkIngredients:
    def dough(self): return "thin crust"
    def sauce(self): return "marinara"
    def cheese(self): return "reggiano"

class ChicagoIngredients:
    def dough(self): return "deep dish"
    def sauce(self): return "plum tomato"
    def cheese(self): return "mozzarella"
```

JAVA

```java
interface IngredientFactory {       // one family
    String dough();
    String sauce();
    String cheese();
}
class NewYorkIngredients
        implements IngredientFactory {
    public String dough()  { return "thin crust"; }
    public String sauce()  { return "marinara"; }
    public String cheese() { return "reggiano"; }
}
class ChicagoIngredients
        implements IngredientFactory {
    public String dough()  { return "deep dish"; }
    public String sauce()  { return "plum tomato"; }
    public String cheese() { return "mozzarella"; }
}
```

INTENT

Create families of related objects without naming their classes.

IN THE REPO

IIngredientFactory, with New York, Chicago and California versions.

JAVA

An interface plus one class per region: the same shape, more braces.

Abstract Factory: the client

CREATIONAL

PYTHON

```python
class CheesePizza:
    def __init__(self, f: IngredientFactory) -> None:
        self._f = f                  # injected family
    def prepare(self) -> list[str]:
        f = self._f                  # a matching set
        return [f.dough(), f.sauce(), f.cheese()]

print(CheesePizza(ChicagoIngredients()).prepare())
# → ['deep dish', 'plum tomato', 'mozzarella']
print(CheesePizza(NewYorkIngredients()).prepare())
# → ['thin crust', 'marinara', 'reggiano']
```

JAVA

```java
class CheesePizza {
    private final IngredientFactory f;  // injected
    CheesePizza(IngredientFactory f) { this.f = f; }
    List<String> prepare() {        // matching set
        return List.of(f.dough(), f.sauce(),
                       f.cheese());
    }
}

// in main():
new CheesePizza(new ChicagoIngredients()).prepare();
// → [deep dish, plum tomato, mozzarella]
new CheesePizza(new NewYorkIngredients()).prepare();
// → [thin crust, marinara, reggiano]
```

GUARANTEE

A Chicago pizza can’t get New York sauce: one factory, one matching set.

MECHANISM

The factory is passed in: composition plus Dependency Inversion.

SWAP IT

Change the factory and every ingredient changes together.

Adapter

STRUCTURAL

PYTHON

```python
class Character(Protocol):    # what the game wants
    def chase(self) -> int: ...
    def flee(self) -> int: ...
    def attack(self) -> int: ...

class Moose:                  # what we actually have
    def run(self) -> int: return 5
    def charge(self) -> int: return 10

class MooseAdapter:           # translates the calls
    def __init__(self, moose: Moose) -> None:
        self._m = moose
    def chase(self) -> int: return self._m.run()
    def flee(self) -> int: return self._m.run()
    def attack(self) -> int: return self._m.charge()

m: Character = MooseAdapter(Moose())
print(m.chase(), m.flee(), m.attack())   # 5 5 10
```

JAVA

```java
interface GameCharacter { // what the game wants
    int chase(); int flee(); int attack();
}
class Moose {             // what we actually have
    int run() { return 5; }
    int charge() { return 10; }
}
class MooseAdapter implements GameCharacter {
    private final Moose m;
    MooseAdapter(Moose m) { this.m = m; }
    public int chase()  { return m.run(); }
    public int flee()   { return m.run(); }
    public int attack() { return m.charge(); }
}

GameCharacter c = new MooseAdapter(new Moose());
// c.chase(), c.flee(), c.attack()  → 5, 5, 10
```

INTENT

Make an existing class fit the interface a client expects.

HOW

Wrap it and translate each call: chase() becomes run().

NAMING

Java already has java.lang.Character, so here it's GameCharacter.

Decorator: one wrapper

STRUCTURAL

PYTHON

```python
class BaseCar:
    drive = attack = defend = 100

class ArmoredCar:                    # wraps ANY car
    def __init__(self, car) -> None:
        self._car = car
    @property
    def drive(self) -> int:
        return self._car.drive - 20
    @property
    def attack(self) -> int:
        return self._car.attack
    @property
    def defend(self) -> int:
        return self._car.defend + 40

armored = ArmoredCar(BaseCar())
print(armored.drive, armored.defend)    # 80 140
```

JAVA

```java
interface Car {
    int drive(); int attack(); int defend();
}
record BaseCar() implements Car {
    public int drive()  { return 100; }
    public int attack() { return 100; }
    public int defend() { return 100; }
}
record ArmoredCar(Car car) implements Car { // wraps
    public int drive()  { return car.drive() - 20; }
    public int attack() { return car.attack(); }
    public int defend() { return car.defend() + 40; }
}

// in main():
Car armored = new ArmoredCar(new BaseCar());
// armored.drive(), armored.defend()  → 80, 140
```

INTENT

Add behavior by wrapping an object in another with the same interface.

PYTHON

@property makes armored.drive a computed attribute.

JAVA

A record holding the car it wraps keeps each wrapper short.

Decorator: stacking wrappers

STRUCTURAL

PYTHON

```python
class RaceCar:                          # another wrapper
    def __init__(self, car) -> None:
        self._car = car
    @property
    def drive(self) -> int:
        return self._car.drive + 40
    @property
    def attack(self) -> int:
        return self._car.attack - 10
    @property
    def defend(self) -> int:
        return self._car.defend - 10

# Wrappers stack: read it inside out
car = RaceCar(ArmoredCar(ArmoredCar(BaseCar())))
print(car.drive, car.attack, car.defend)
# → 100 90 170
```

JAVA

```java
record RaceCar(Car car) implements Car { // wraps too
    public int drive()  { return car.drive() + 40; }
    public int attack() { return car.attack() - 10; }
    public int defend() { return car.defend() - 10; }
}

// in main(): wrappers stack, read inside out
Car car = new RaceCar(new ArmoredCar(
    new ArmoredCar(new BaseCar())));
// car.drive(), car.attack(), car.defend()
// → 100, 90, 170
```

STACKABLE

A wrapped car is still a car, so wrappers nest: armor twice, then race.

NO EXPLOSION

Without the pattern you'd need a subclass for every combination.

NOT @DECORATOR

Python's @ syntax wraps functions: the same idea, a different feature.

Facade: the subsystem

STRUCTURAL

PYTHON

```python
class Confusing:                    # awkward, as found
    def execute(self, a, b, c): return a + b + c
    def method1(self): return 0
    def method2(self, x, y=0): return x + y

class Overdone:
    def __init__(self, some_string): ...
    def do_something(self, x, y):
        return x * y
    def do_something_else(self, x, y, z):
        return (x + y) * z
    def do_something_again(self, x, y, z, a):
        return (x + y + z) * a
```

JAVA

```java
class Confusing {                       // awkward, as found
    int execute(int a, int b, int c) {
        return a + b + c; }
    int method1() { return 0; }
    int method2(int x, int y) { return x + y; }
}
class Overdone {
    Overdone(String someString) {}
    int doSomething(int x, int y) { return x * y; }
    int doSomethingElse(int x, int y, int z) {
        return (x + y) * z;
    }
    int doSomethingAgain(int x, int y, int z,
                         int a) {
        return (x + y + z) * a;
    }
}
```

AS FOUND

The repo names its subsystem honestly: Confusing and Overdone.

THE PROBLEM

Callers must know which class does what, and which method to call.

NEXT

A facade gives them one simple front door instead.

Facade: the front door

STRUCTURAL

PYTHON

```python
class BetterApi:                        # the facade
    def __init__(self, confusing=None,
                 overdone=None):
        self._c = confusing or Confusing()
        self._o = overdone or Overdone("foo")

    def add_three_numbers(self, a, b, c):
        return self._c.execute(a, b, c)

    def add_then_multiply(self, *nums, factor):
        calls = {1: self._o.do_something,
                 2: self._o.do_something_else,
                 3: self._o.do_something_again}
        return calls[len(nums)](*nums, factor)

api = BetterApi()
api.add_then_multiply(4, 3, factor=5)   # 35
```

JAVA

```java
class BetterApi {                       // the facade
    private final Confusing c = new Confusing();
    private final Overdone o = new Overdone("foo");

    int addThreeNumbers(int a, int b, int x) {
        return c.execute(a, b, x);
    }
    // Java has real overloads, like the repo's C#:
    int addThenMultiply(int a, int f) {
        return o.doSomething(a, f);
    }
    int addThenMultiply(int a, int b, int f) {
        return o.doSomethingElse(a, b, f);
    }
    int addThenMultiply(int x, int y, int z, int f) {
        return o.doSomethingAgain(x, y, z, f);
    }
}
```

INTENT

Callers see two well-named methods and never touch the subsystem.

PYTHON

No overloads, so *nums and a dict pick the right call.

JAVA

Real overloads, as in the repo’s C#: one method per argument count.

Strategy: the powers

BEHAVIORAL

PYTHON

```python
class SuperPower(Protocol):
    def exercise_power(self) -> str: ...

class Fly:
    def exercise_power(self) -> str:
        return "Up, Up, and Away!"

class WeaveWeb:
    def exercise_power(self) -> str:
        return "Catches Thieves Like Flies!"

# Each power is a small, swappable object.
# Python could also just pass a function.
```

JAVA

```java
interface SuperPower {
    String exercisePower();
}

class Fly implements SuperPower {
    public String exercisePower() {
        return "Up, Up, and Away!";
    }
}

class WeaveWeb implements SuperPower {
    public String exercisePower() {
        return "Catches Thieves Like Flies!";
    }
}

// A one-method interface also takes a lambda:
SuperPower fight = () -> "KaBaam!";
```

INTENT

Put each algorithm behind one interface and make them swappable.

PYTHON

A Protocol; each power is a small class with one method.

JAVA

An interface; with one method, a lambda can be a power too.

Strategy: the hero

BEHAVIORAL

PYTHON

```python
class Hero:
    def __init__(self, power: SuperPower) -> None:
        self._power = power
    def do_hero_stuff(self) -> str:
        return self._power.exercise_power()
    def change_super_power(self, power: SuperPower):
        self._power = power         # swap at runtime

class SpiderMan(Hero):
    def __init__(self): super().__init__(WeaveWeb())

spidey = SpiderMan()
spidey.do_hero_stuff() # Catches Thieves Like Flies!
spidey.change_super_power(Fly())
spidey.do_hero_stuff() # Up, Up, and Away!
```

JAVA

```java
class Hero {
    private SuperPower power;
    Hero(SuperPower power) { this.power = power; }
    String doHeroStuff() {
        return power.exercisePower();
    }
    void changeSuperPower(SuperPower power) {
        this.power = power;     // swap at runtime
    }
}
class SpiderMan extends Hero {
    SpiderMan() { super(new WeaveWeb()); }
}

Hero spidey = new SpiderMan();
spidey.doHeroStuff(); // Catches Thieves Like Flies!
spidey.changeSuperPower(new Fly());
spidey.doHeroStuff(); // Up, Up, and Away!
```

DELEGATE

do_hero_stuff() hands the work to whatever power the hero holds.

RUNTIME

change_super_power() swaps the behavior of a live object.

PRINCIPLES

Open/Closed: a new power is a new class; Hero never changes.

Observer: the subject

BEHAVIORAL

PYTHON

```python
@dataclass(frozen=True)
class GameResult:
    winner: str
    loser: str
    score: str

class Observer(Protocol):
    def update(self, r: GameResult) -> None: ...
class SportsAggregator:                  # the subject
    def __init__(self) -> None:
        self._observers: list[Observer] = []
    def register(self, o: Observer) -> None:
        self._observers.append(o)
    def unregister(self, o: Observer) -> None:
        self._observers.remove(o)
    def add_game_result(self, r: GameResult):
        for o in list(self._observers):
            o.update(r)              # push to everyone
```

JAVA

```java
record GameResult(String winner, String loser,
                  String score) {}    // immutable
interface Observer {
    void update(GameResult r);
}
class SportsAggregator {              // the subject
    private final List<Observer> observers =
        new ArrayList<>();
    void register(Observer o) { observers.add(o); }
    void unregister(Observer o) {
        observers.remove(o);
    }
    void addGameResult(GameResult r) {
        for (Observer o : List.copyOf(observers)) {
            o.update(r);          // push to everyone
        }
    }
}
```

INTENT

One subject, many observers: each change is pushed to everyone registered.

IMMUTABLE DATA

A frozen dataclass in Python is a record in Java.

SAFE LOOP

Both loop over a copy, so an observer can unregister mid-loop.

Observer: the observers

BEHAVIORAL

PYTHON

```python
class NewsPaper:
    def __init__(self) -> None:
        self.results: list[GameResult] = []
    def update(self, result: GameResult) -> None:
        self.results.append(result)

class RadioStation:
    def update(self, result: GameResult) -> None:
        print(f"Breaking: {result.winner}")

hub = SportsAggregator()
paper, radio = NewsPaper(), RadioStation()
hub.register(paper); hub.register(radio)
hub.add_game_result(
    GameResult("Lions", "Eagles", "2-1"))
# radio prints 'Breaking: Lions'
# Repo's weak-reference variant, in Python:
#   self._observers = weakref.WeakSet()
```

JAVA

```java
class NewsPaper implements Observer {
    List<GameResult> results = new ArrayList<>();
    public void update(GameResult r) {
        results.add(r);
    }
}
class RadioStation implements Observer {
    public void update(GameResult r) {
        System.out.println(
            "Breaking: " + r.winner());
    }
}
var hub = new SportsAggregator();
var paper = new NewsPaper(); hub.register(paper);
hub.register(new RadioStation());
hub.addGameResult(
    new GameResult("Lions", "Eagles", "2-1"));
//> Breaking: Lions
```

OPEN/CLOSED

Adding a TV station needs no change to the aggregator.

JAVA BONUS

Observer has one method, so a lambda can subscribe directly.

IN THE REPO

A weak-reference variant too: weakref.WeakSet in Python.

Command: the command

BEHAVIORAL

PYTHON

```python
class Command(Protocol):
    def execute(self) -> None: ...
    def undo(self) -> None: ...

class Document:                         # the receiver
    def __init__(self, text: str = "") -> None:
        self.text = text

class AddText:                  # request as an object
    def __init__(self, doc: Document, text: str):
        self.doc, self.text = doc, text
    def execute(self) -> None:
        self.doc.text += self.text
    def undo(self) -> None:
        d = self.doc
        d.text = d.text.removesuffix(self.text)
```

JAVA

```java
interface Command { void execute(); void undo(); }
class Document {                        // the receiver
    String text;
    Document(String text) { this.text = text; }
}

class AddText implements Command { // a request
    private final Document doc;
    private final String text;
    AddText(Document doc, String text) {
        this.doc = doc; this.text = text;
    }
    public void execute() { doc.text += text; }
    public void undo() {
        doc.text = doc.text.substring(
            0, doc.text.length() - text.length());
    }
}
```

INTENT

Turn a request into an object, so it can be queued, logged or undone.

RECEIVER

Document is what changes; AddText knows how to do and undo it.

STRINGS

Immutable in both languages: undo builds a new, shorter string.

Command: the invoker

BEHAVIORAL

PYTHON

```python
class Editor:                          # the invoker
    def __init__(self) -> None:
        self.history: list[Command] = []
    def run(self, cmd: Command) -> None:
        cmd.execute()
        self.history.append(cmd)
    def undo(self) -> None:
        self.history.pop().undo()

doc, editor = Document("Patterns"), Editor()
editor.run(AddText(doc, " are fun"))
doc.text        # 'Patterns are fun'
editor.undo()
doc.text        # 'Patterns'
```

JAVA

```java
class Editor {                          // the invoker
    private final Deque<Command> history =
        new ArrayDeque<>();
    void run(Command cmd) {
        cmd.execute();
        history.push(cmd);
    }
    void undo() { history.pop().undo(); }
}

var doc = new Document("Patterns");
var editor = new Editor();
editor.run(new AddText(doc, " are fun"));
// doc.text → "Patterns are fun"
editor.undo();
// doc.text → "Patterns"
```

INVOKER

The editor runs commands and keeps a history, knowing only Command.

UNDO

Pop the last command and call its undo().

EXTEND

A new command, such as AddNumber, gets undo for free.

Template Method: the skeleton

BEHAVIORAL

PYTHON

```python
class PizzaStore(ABC):
    def create_pizza_for_delivery(self) -> list[str]:
        return [self.take_order(),     # the template
                self.process_payment(),
                self.make_pizza(),
                self.cook_pizza(),
                self.deliver_pizza()]

    def process_payment(self): return "paid"
    def deliver_pizza(self): return "delivered"

    @abstractmethod
    def take_order(self) -> str: ...
    @abstractmethod
    def make_pizza(self) -> str: ...
    @abstractmethod
    def cook_pizza(self) -> str: ...
```

JAVA

```java
abstract class PizzaStore {
    final List<String> createPizzaForDelivery() {
        return List.of(takeOrder(),  // the template
                       processPayment(),
                       makePizza(),
                       cookPizza(),
                       deliverPizza());
    }

    String processPayment() { return "paid"; }
    String deliverPizza() { return "delivered"; }

    abstract String takeOrder();
    abstract String makePizza();
    abstract String cookPizza();
}
```

INTENT

The base class fixes the steps; subclasses fill in the ones that vary.

GOOD INHERITANCE

A shallow tree, and the parent owns the flow.

JAVA EXTRA

final stops overriding the template; Python's @final only warns via mypy.

Template Method: one store

BEHAVIORAL

PYTHON

```python
class NewYorkStylePizzaStore(PizzaStore):
    def take_order(self) -> str:
        return "1 large, thin crust"
    def make_pizza(self) -> str:
        return "hand-tossed dough"
    def cook_pizza(self) -> str:
        return "deck oven, 6 min"

store = NewYorkStylePizzaStore()
for step in store.create_pizza_for_delivery():
    print(step)
# -> 1 large, thin crust
#    paid
#    hand-tossed dough
#    deck oven, 6 min
#    delivered
```

JAVA

```java
class NewYorkStylePizzaStore extends PizzaStore {
    String takeOrder() {
        return "1 large, thin crust";
    }
    String makePizza() {
        return "hand-tossed dough";
    }
    String cookPizza() {
        return "deck oven, 6 min";
    }
}
var store = new NewYorkStylePizzaStore();
for (String step : store.createPizzaForDelivery())
    System.out.println(step);
// -> 1 large, thin crust | paid | hand-tossed
//    dough | deck oven, 6 min | delivered
```

THE SUBCLASS

A store supplies just the three varying steps.

IN THE REPO

BasePizzaStore.CreatePizzaForDelivery() and a New York store.

VS STRATEGY

Template Method varies steps by subclassing; Strategy by a held object.

How the patterns reuse behavior

PUTTING IT TOGETHER

Pattern	Reuses behavior by	Serves
Strategy	composition: holds a swappable object	OCP, DIP
Decorator	composition: wraps the same interface	OCP, SRP
Adapter	composition: wraps a foreign interface	DIP, ISP
Facade	composition: hides a subsystem	SRP, DIP
Observer	composition: a list of subscribers	OCP, DIP
Command	composition: requests as objects	SRP, OCP
Abstract Factory	composition: an injected product family	DIP, OCP
Factory & Template Method	inheritance: a subclass fills a hook	OCP
Singleton	neither: one global instance	works against DIP

When the language makes a pattern disappear

SHORTCUTS

Pattern	In Python	In modern Java	Keep the class when…
Strategy	pass a function	a lambda	the strategy has state or several methods
Command	a callable, functools.partial	a Runnable lambda	you need undo, history or a queue
Singleton	a module, or @functools.cache	an enum	almost never: inject it instead
Observer	a list of callbacks	a list of Consumer<T>	observers need several hooks
Simple Factory	a dict of classes	Map<K, Supplier<T>>	creation needs real logic
Iterator	a generator with yield	Iterable, Stream	rarely

Peter Norvig (1996): 16 of the 23 GoF patterns become simpler or invisible in dynamic languages.

When patterns hurt

PITFALLS

Pattern before problem

An abstraction with one use. Wait for the second or third case.

Singleton as a global

Hidden dependencies and tests that leak state into each other.

An interface for every class

Add the abstraction when the second implementation arrives.

Deep hierarchies

Five levels of base classes: a change at the top breaks the bottom.

Every pattern buys flexibility with indirection. Pay only when you need it.

Five things to take back to your code

WRAP-UP

1. Prefer composition. Inherit only for a true, substitutable is-a.
2. SOLID has one goal: make the next change small and local.
3. Depend on interfaces, and build concrete objects at the edge of the app.
4. Patterns are vocabulary. Reach for one when its problem shows up.
5. Try the simplest form first: a function, a lambda, a module, a map.

Hands-on lab

≈ 30 MIN · PAIRS

DECORATOR

Add a ShieldCar (+60 defend, −30 drive) and stack it on a RaceCar.

STRATEGY

Let a hero hold several powers, and give SpiderMan Fly too.

ADAPTER

Write a DragonAdapter so a Dragon can join the game as a Character.

OBSERVER

Make register() return an unsubscribe function, then test it.

COMMAND

Add an AddNumber command and a redo() method on Editor.

DIP

Write a test for OrderService using InMemoryRepo: no database.

Pick Python or Java. The companion folder has both; keep pytest -q green after every exercise.

Further reading

REFERENCES

Design Patterns: Elements of Reusable Object-Oriented Software

Gamma, Helm, Johnson, Vlissides (1994): the original catalogue of 23

Head First Design Patterns

Freeman and Robson: the pizza-store style of example used in the repo, in Java

Effective Java, 3rd edition

Joshua Bloch (2018): Java idioms such as the enum singleton and the holder class

Refactoring.Guru: design patterns

Illustrated explanations, with code in Python, Java and more

typing.Protocol

Python docs: structural subtyping (PEP 544)

github.com/himasz/design-patterns-c-sharp

The C# originals for every example; samples by Philip Japikse, MIT licence

THANK YOU

Questions?

Every example runs: the Python on 3.10+ with a pytest suite, the Java on Java 21.